Lab 2 - 2: Automating Nightly Uploads (Using AWS CloudShell)

1. Introduction & Scenario

Welcome back, Data Manager! In the last lab, you used the AWS Console to store Project Starlight's first image. It worked perfectly, but the telescope is now capturing new data every night. Clicking and dragging files into a web browser isn't a scalable solution. It's time to learn how cloud professionals automate their work: with code.

In this lab, you will perform the **exact same tasks** as in Lab 1—creating a bucket and uploading a file—but this time you will use **AWS CloudShell**. CloudShell is a command-line interface (CLI) that runs directly in your browser. This will powerfully demonstrate how any action in the console can be done faster and more efficiently with commands.

2. Learning Objectives

By the end of this lab, you will be able to:

- Launch and use the AWS CloudShell environment.
- Upload a local file into the CloudShell environment.
- Use basic S3 commands (`mb`, `cp`, `ls`, `rm`, `rb`) to manage buckets and objects.
- Understand the relationship between console actions and their command-line equivalents.

3. Prerequisites

- An AWS account with IAM user access to the AWS Management Console.

---

4. Lab Steps

Part A: Launch AWS CloudShell

First, we need to open our command-line environment.

1. Sign in to the **AWS Management Console**.
2. In the top-right menu bar, look for the CloudShell icon. It looks like a small terminal window `>_`. Click it.
3. A new panel will open at the bottom of your screen. Wait a moment for the environment to be prepared. You will see a command prompt when it's ready, which looks something like `[cloudshell-user@ip-10-0-1-100 ~]$`.

You are now inside a fully functional Linux terminal that is already authenticated to your AWS account!

Part B: Upload Your Image to CloudShell

Your CloudShell terminal is separate from your local computer. Before you can upload the image to S3, you must first get it from your desktop *into* CloudShell.

Your CloudShell terminal is separate from your local computer. Instead of uploading the file from your desktop, we'll use the command line to download it directly from the internet into our CloudShell environment.

1. First, let's create a dedicated directory to keep our project files organized. Use the mkdir command, which stands for "make directory".

   ```
   mkdir my_project
   ```

2. Now, move into that new directory. Use the cd command, which stands for "change directory".

   ```
   cd my_project
   ```

   Your command prompt should now show that you are inside the my_project directory.

3. Finally, download the image using wget, a command that retrieves content from web servers. Replace the placeholder URL with the actual link to the image.

   ```
   wget https://cdn.esahubble.org/archives/images/publicationjpg/heic1502a.jpg -O andromeda.jpg
   ```

   The -O andromeda.jpg part of the command renames the downloaded file to andromeda.jpg for simplicity.

4. To verify the download was successful, use the ls command (for "list") to see the contents of your current directory.

   ```
   ls
   ```

   You should see andromeda.jpg listed in the output.

5. You can change your filename with mv command if you decided to change for any reason

   ```
   mv old-name new-name
   ```

Part C: Create Your Bucket (with code)

Now, let's create our S3 bucket using a command.

1. First, let's define a variable to hold our unique bucket name. This saves us from retyping it and making errors. **Remember to change yourname and the date!**

   ```
   BUCKET_NAME="starlight-code-yourname-20250917"
   ```

2. Now, use the aws s3 mb command (mb stands for "make bucket") to create the bucket.

   ```
   aws s3 mb s3://$BUCKET_NAME
   ```

3. The command will use the default region for your CloudShell session. You should see a success message like make_bucket: starlight-code-yourname....

Part D: Create a Folder Structure (with code)

In S3, folders don't truly exist until an object is placed inside them. The "folder path" is just part of the object's name, called a **prefix**. The easiest way to create a folder with the CLI is to upload a placeholder file into it.

1. We'll use the aws s3 cp command (cp stands for "copy") to upload our image directly into the raw-images/ prefix. This single command both uploads the file and creates the "folder" if it doesn't exist.

   ```
   aws s3 cp andromeda.jpg s3://$BUCKET_NAME/raw-images/
   ```

2. You should see a confirmation message: `upload: ./andromeda.jpg to s3://starlight-code.../raw-images/andromeda.jpg`

Part E: Verify Your Upload (with code)

How do we check that the file is really there? We use the `aws s3 ls` command (`ls` for "list").

1. Let's list the contents of our bucket. The `--recursive` flag tells the command to show the contents of all folders.

   ```
   aws s3 ls s3://$BUCKET_NAME --recursive
   ```

2. The output will show your `raw-images/andromeda.jpg` object, confirming the upload was successful and the folder structure is correct.

Deliverable

To receive completion marks for this lab, you must submit a screenshot of your CloudShell window. Your screenshot must clearly show the successful output of the verification command from Part E, aws s3 ls s3://$BUCKET_NAME --recursive. This command output proves you created the bucket and uploaded the file to the correct location using the CLI.

Part F: Manage and Delete the Object (with code)

Just like in the console, we can also delete objects.

1. Use the `aws s3 rm` command (`rm` for "remove") to delete the image.

   ```
   aws s3 rm s3://$BUCKET_NAME/raw-images/andromeda.jpg
   ```

You'll see a confirmation message: `delete: s3://starlight-code.../raw-images/andromeda.jpg`

Part G: Cleanup (with code)

Finally, let's remove the bucket we created. A bucket must be empty before it can be deleted.

1. Use the `aws s3 rb` command (`rb` for "remove bucket") to delete the now-empty bucket.

   ```
   aws s3 rb s3://$BUCKET_NAME
   ```

   You will see the final confirmation: `remove_bucket: starlight-code...`

Fantastic work! You have now completed the exact same workflow from Lab 1, but entirely with commands. You can see how typing a few lines is much faster than clicking through multiple screens. This efficiency is the foundation of cloud automation. In the next step, you'll use these new command-line skills to complete your first assignment.